

ONYX & XELOR

MVP — technology and architecture

What the two products are built from **today**, how a request travels through them, how the AI is wired in, and how the nine agents are connected into graphs that run, pause for a human, and close with evidence. The companion document sets out what this becomes as a production product.

01 One codebase, two products

XELOR is not a fork of ONYX and does not replace it. It reads ONYX's records and writes back through ONYX's own endpoints, so the two run side by side on separate ports and separate databases from a shared pnpm workspace.

	ONYX BY AIKYANTRA — PHASE 1	XELOR BY AIKYANTRA — PHASE 2
Role	The system of record: sales, purchase, inventory, production, quality, maintenance, people, accounts.	The intelligence layer: fulfilment missions, the nine agents, the copilot, the AI control centre.
Runs on	api :3000 · web :3001 · db indcore	api :3100 · web :3101 · db indcore_p2
API modules	21 (accounts ... workflow)	22 — adds <code>dataimport</code> , plus the top-level <code>fulfilment</code> module
Shared domain	<code>packages/platform</code> — framework-free contracts and rules. XELOR adds <code>fulfilment</code> , <code>factory-intelligence</code> , <code>intelligence</code> , <code>dataimport</code> ; ONYX carries <code>factory-operations</code> instead.	
Migrations	92 SQL files	96 SQL files — a superset, which is why the databases are never shared
Shared services	Keycloak (one realm, one sign-in theme), Valkey and Gotenberg — none hold phase-specific state.	

02 The stack, by layer

LAYER	TECHNOLOGY	WHAT IT DOES HERE
Runtime	Node 22 · TypeScript 5.7 · pnpm workspace	ESM throughout; one lockfile, three apps, two shared packages.
API	NestJS 11 · Express 5 · Zod 3	Modular monolith behind the prefix <code>api/v1</code> . Zod validates every request body and AI response.
Web	Next.js 15 · React 19 · Tailwind 4 · shadcn/ui on Radix · lucide-react · three.js	App Router with server components; three.js draws the factory floor only.
Database	PostgreSQL 17 (pgvector/pgvector:pg17) · Drizzle ORM 0.36	One shared schema, multi-tenant by <code>FORCE ROW LEVEL SECURITY</code> ; pgvector backs retrieval.
Identity	Keycloak 26 · OIDC · jose (JWT verify)	One realm for both products; the API verifies audience and issuer, then resolves tenant and permissions.
Async	Valkey 8 · BullMQ 5 · ioredis	Outbox → relay → queue → idempotent consumer. Never carries ledger-critical writes.
AI	Provider-agnostic router · Ollama (local) · offline stub · pgvector	One doorway, a closed feature registry, a hash-chained call log. No business module imports a provider SDK.
Documents	Gotenberg 8 · Playwright/Chromium · SheetJS (xlsx)	Invoices and packs as PDF; spreadsheets through the data-import module.
Machines	apps/edge — simulator · OPC UA · MQTT · ROS 2 · vendor adapters	A runtime beside the line: reads state, executes one named, time-boxed capability at a time.
Infra	Docker Compose · Caddy · Railway (demo) · AWS ap-south-1 (target)	Four containers locally: Postgres, Valkey, Keycloak, Gotenberg.
Proof	node:test · Playwright e2e · ESLint boundary rules · GitHub Actions	Every merge must pass build, boundary lint, typecheck, migrations, unit tests, the AI eval and grounding gates, and the RLS, permission, naming and module-manifest proofs.

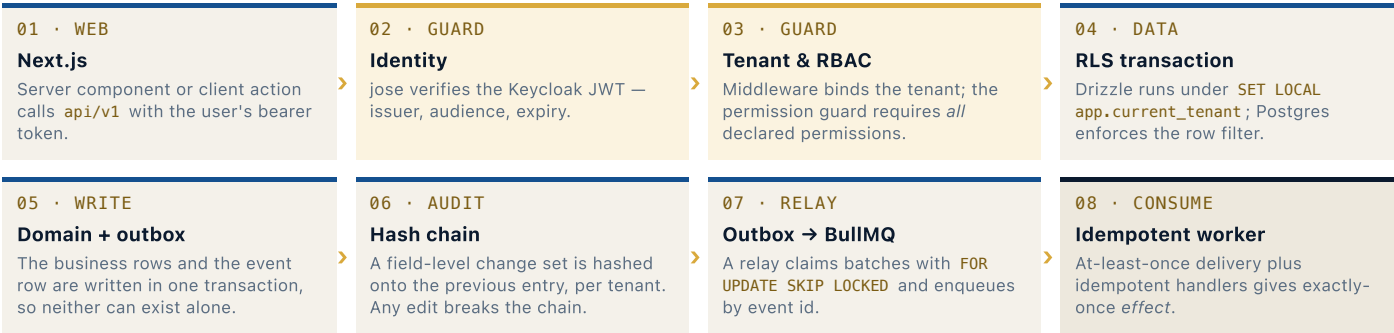
03 Repository shape

APPS/API modules/ one folder per business domain · agent-os/ the agent runtime · ai/ router and features · bus/ outbox relay and queues · ports/ the only legal cross-module doorways · fulfilment/ missions, XELOR only.	APPS/WEB modules/ mirrors the API one-for-one, assembled by a single <code>registry.ts</code> · spine/ shell, navigation and copilot driver · scripts/ the PDF render pipeline.	PACKAGES/ platform — domain logic with no NestJS and no database: registries, graph validation, hash chains, tax, planning, confidence. db — Drizzle schema, migrations, tenant-scoped connections.
---	---	---

Module boundaries are a lint rule, not a convention — `eslint-plugin-boundaries` fails CI. No module imports another's service and no foreign key crosses a boundary. Everything goes through a typed port — `STOCK`, `BOM`, `WORKFLOW`, `CUSTOMER_ORDER_WRITER` — or a versioned event. Production may never write stock: the single write path `POST /api/v1/stock/entries` belongs to Inventory alone.

04 How one request moves

Every write in both products follows the same eight steps. Nothing skips a step, including anything an agent initiates — an agent is just another authenticated caller.



THE WRITE CONTRACT

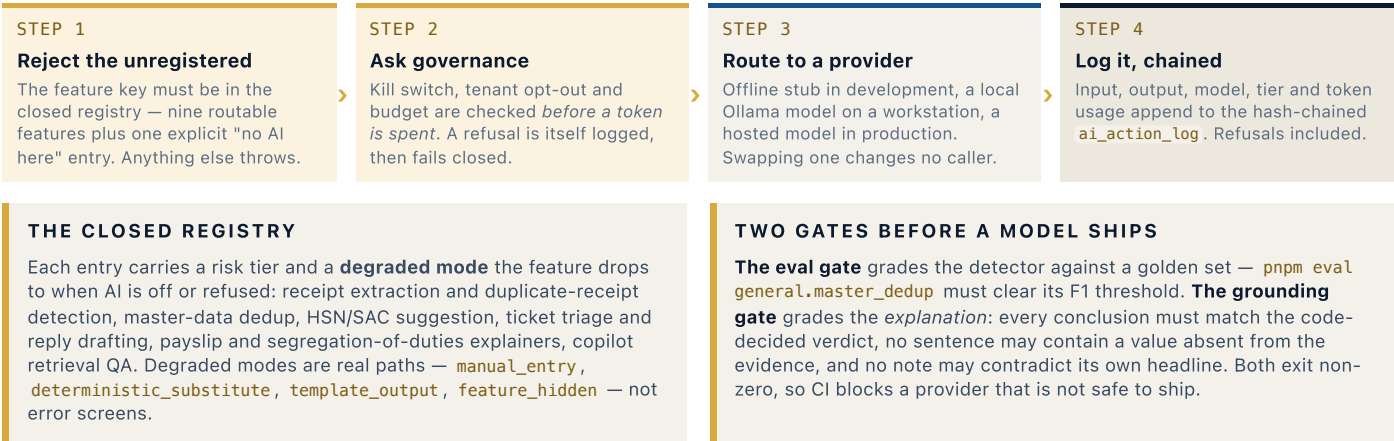
UUIDv7 primary keys · `tenant_id` on every tenant-scoped table with composite indexes leading on it · `created_at/by`, `updated_at/by`, `is_active` soft delete · no hard delete on master, financial or statutory rows · money as `NUMERIC(18,2)` · effective-dated rate masters · one canonical error envelope · cursor pagination only · `Idempotency-Key` on every mutating endpoint.

THE EVENT CONTRACT

Versioned names — `module.entity.verb.v1` — published only through the transactional outbox. The relay drains per tenant under that tenant's RLS context, so the bus never becomes a way around the row filter. **Ledger-critical writes never travel on the bus:** they stay synchronous inside the one transaction that produced them.

05 How the AI is connected

No business module ever talks to a model. Every call passes through one router, which does exactly four things before and after the provider — and contains no business logic.



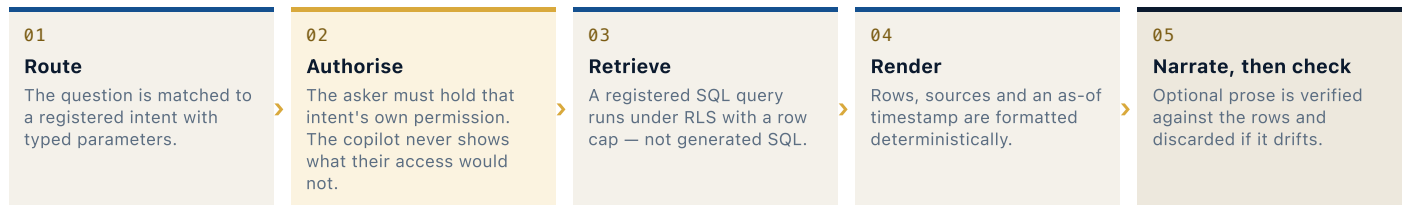
06 The nine agents

Registered in code, not configured at runtime: each may invoke only capabilities matching a prefix it owns.

AGENT	DEPARTMENT	OWNS	CAPABILITY PREFIXES
ONYX	Mission Control	Accepts the goal, coordinates specialists, synthesises the outcome — the only agent that delegates.	<code>agent.workflow.general</code> .
HEXA	Platform & Governance	Policy, authorization, budgets, approvals, auditability; verifies boundaries.	<code>governance.agent.workflow.general</code> .
MICA	Sales & Product Care	Customers, quotations, orders, delivery commitments, complaints, warranty.	<code>sales.customer.agent</code> .
SPAR	Inventory & Procurement	Stock, shortages, suppliers, procurement options.	<code>inventory.purchase.supplier.agent</code> .
AXLE	Engineering & Planning	BOMs, routings, material plans, capacity, schedules.	<code>engineering.planning.agent</code> .
KILN	Production, Quality & Maintenance	Execution, quality evidence, audit readiness, CAPA, shop-floor risk.	<code>production.quality.maintenance.agent</code> .
RASP	Finance & People	Cash, collections, margin, controls, workforce and payroll.	<code>accounts.finance.expenditure.hrm.agent</code> .
RELAY	Managed Services	Service lifecycle, event triage, incidents, service levels, customer updates.	<code>managed-services.agent</code> .
ACHILES	Platform Health	The platform's own health and boundary — it watches XELOR itself.	<code>platform-health</code> .

07 The copilot pipeline

The copilot is retrieval-first and deterministic: the model is the last step, not the first, and it may rephrase — never compute.



08 The graph, and the loop that walks it

A graph is an auditable execution plan, not a prompt. Eight are registered — foundation mission, factory flow and factory intelligence recovery, full command review, controlled action, working-capital review, QMS audit readiness, managed-service assurance. A model may supply a node's structured output; it can never invent a node, an edge, a permission or a side effect.

SIX NODE KINDS

agent reasons · **capability** calls one of 20 registered tools · **transform** joins evidence · **branch** chooses on a condition · **approval** stops for a person · **verification** re-checks the boundary. Every capability is graded **read**, **analyse**, **simulate**, **draft** or **execute**.

VALIDATED BEFORE IT RUNS

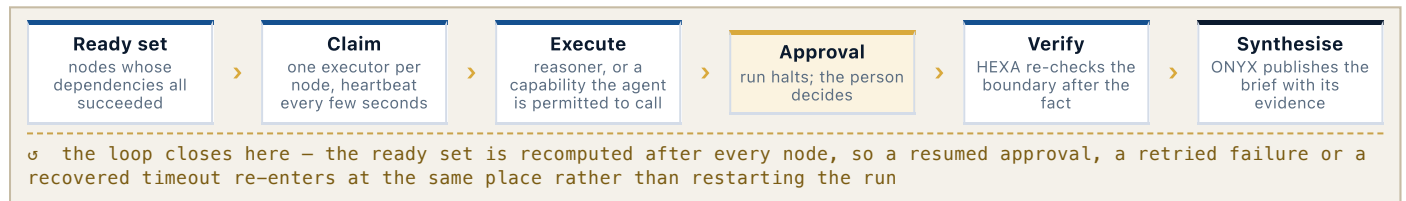
validateAgentGraph rejects a cycle, a dangling dependency, a branch with no source, or a node asking for more than five attempts. Each node carries its own status — pending, running, waiting_approval, succeeded, failed, skipped, cancelled — so a run is inspectable at any moment.

A CAPABILITY IS A CONTRACT, NOT A CALL

Each of the 20 declares the permission it needs, the agents allowed to hold it, a Zod parser for its input, whether it has side effects, whether it needs approval, and an execution boundary of **domain_service** — so it reaches data through the very same module service a person's request would, never through SQL of its own.

AN AGENT RUNS AS THE PERSON WHO LAUNCHED IT

Internal tool calls skip the controller guard, so the RBAC check is repeated inside the runtime against the operator's own roles, under their tenant's RLS context. **An agent can narrow a user's authority; it can never widen it.**



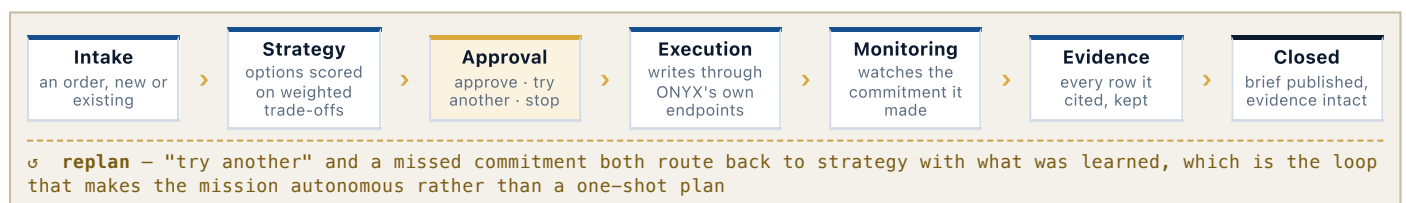
TWO REFUSALS, TWO BEHAVIOURS

An operator lacking a permission (**AGENT_TOOL_FORBIDDEN**) only skips that node — people legitimately see different things. A graph naming a capability its agent may *never* hold (**AGENT_CAPABILITY_FORBIDDEN**) fails the run loudly: a static misconfiguration, identical for everyone.

CONFIDENCE RATES EVIDENCE, NOT ODDS

Every decision carries a deterministic score across evidence coverage, freshness, completeness and learning history, with its strengths and gaps named. It never claims a probability that the proposed action will work.

09 The fulfilment mission arc, and the reach to the machine



DOWN TO THE LINE

apps/edge runs beside the machines behind adapters for simulator, OPC UA, MQTT, ROS 2 and vendor controllers. It reads state — running, idle, blocked, faulted, protective stop, offline — and executes exactly six capabilities: enqueue a robot job, select an approved program, pause after cycle, dispatch an AMR route, quarantine output, request an inspection.

BOUNDED BY CLOCKS, NOT BY TRUST

Commands are fingerprinted with SHA-256 and single-flighted, so a duplicate key cannot fire twice. A command expires in 15 minutes, telemetry older than 2 minutes is stale, and an approval older than 15 minutes authorises nothing. Safety is asked of the local adapter immediately before execution, never inferred.

The rule the whole architecture exists to keep: AI explains, it never decides. Nothing reaches a customer, a supplier, a ledger or a machine without a person approving it, and every approval is reconstructable afterwards — the cited rows, the model call, the permission held, and the hash chain proving none of it was edited since.